ENGINEERING BLUEPRINT & EXECUTION SPECIFICATION

AeroMESH Windows Desktop Production Engineering Plan

Autonomous Self-Bootstrapping Runtime, Security Hardening, Kernel Lifecycle Trapping, and 3-Step First-Run Onboarding

DOCUMENT VERSION	TARGET PLATFORM	AUTHORING ROLES	BASELINE TEST SUITE
2.0.0 (Commercial Ready)	Windows 10 / Windows 11 (x86_64)	Principal Architect & Security Lead	92/92 Automated Tests Passing

Executive Summary

This specification outlines the definitive engineering roadmap to upgrade AeroMESH into a commercial-grade, self-bootstrapping native Windows desktop application. It remediates all previously identified lifecycle leaks, security vulnerabilities, and permission faults while introducing an autonomous runtime that eliminates all reliance on developer setup scripts (setup.ps1, start.ps1), guarantees zero forced model downloads, and provides a polished Kimi dark matte onboarding wizard with strictly Lucide icons.

1. Master Fixing Strategy & Upgraded System Architecture

The core objective is to transition AeroMESH from a developer script running in an unmanaged Edge browser window into a hardened, self-contained native Windows desktop application.



Strategic Core Pillars

- **Self-Bootstrapping Autonomy:** The binary provisions its own directory hierarchy, generates CSPRNG keys, verifies CUDA runtime DLLs, and resolves network topologies without invoking external shell scripts.
- **Zero Forced Model Downloads:** Unconditional model downloads (e.g. DreamShaper 8 in legacy start.ps1) are completely excised. No weights are transferred without explicit user opt-in.
- **First-Run Onboarding Wizard:** A frictionless 3-step modal flow inside the UI handles diagnostics, opt-in starter models with streaming progress, and role launch.
- **Kernel-Enforced Lifecycle Trapping:** SafeProcessJob binds all child engines (aeromesh.exe, sd-cli.exe) to a Windows Job Object with `JOB_OBJECT_LIMIT_KILL_ON_JOB_CLOSE`.
- **Kimi Dark Matte Aesthetic:** Deep obsidian matte surfaces, liquid glass panels, and strictly Lucide SVG icons (zero emojis).

2. Architectural Solutions & Decision Evaluation Matrices

Decision 1: Window Lifecycle & Process Zombie Remediation

Approach	Technical Concept	Pros & Strengths	Cons & Trade-Offs	Verdict
Option A: Embedded WebView2	Rewrite host using wry/tao embedding WebView2 COM loader directly.	True native HWND window; clean WM_CLOSE events; no external process.	Heavy C++ COM bindings; build friction on MinGW/MSVC; risk to CSS filters.	REJECTED
Option B: Supervised Edge + Job Object	Retain Edge App-Mode; supervise Child handle; trap in SafeProcessJob.	Zero new C++ GUI crates; kernel-guaranteed VRAM release; sub-100ms exit.	Requires Microsoft Edge present on host (standard on 99.8% of Windows).	SELECTED
Option C: Background Tray Daemon	Run perpetual background tray service; closing window minimizes.	Models stay in VRAM between sessions; similar UX to Docker/Ollama.	Confuses users expecting window close to free GPU VRAM for gaming.	REJECTED

Decision 2: File Handling, Storage Locations & Windows Permissions

Approach	Technical Concept	Pros & Strengths	Cons & Trade-Offs	Verdict
Option A: UAC Admin Elevation	Embed requireAdministrator manifest to write directly to Program Files.	Zero path code changes.	Violates enterprise security; blocked by corporate IT; UAC prompt on every run.	REJECTED
Option B: Windows AppData Hierarchy	Map config to %APPDATA%, cache/profiles to %LOCALAPPDATA%.	100% compliant with Windows standards; works under standard user accounts.	Requires centralizing path resolution across CLI and desktop crates.	SELECTED
Option C: MSIX Virtualization	Rely on Windows VFS filesystem redirection in an MSIX package.	No path modifications inside code.	Masks structural flaws; breaks external symlinks; signing friction.	REJECTED

Decision 3: Localhost API Security & CORS Remediation

Approach	Technical Concept	Pros & Strengths	Cons & Trade-Offs	Verdict
Option A: CSPRNG Token + Host Guard	Generate BCrypt token; inject in meta tag; restrict CORS to 127.0.0.1.	Defeats CORS drive-by and DNS rebinding; preserves OpenAI client compatibility.	Frontend fetch calls must attach Authorization header.	SELECTED
Option B: Ephemeral Local mTLS	Generate self-signed certificate on startup; bind HTTPS loopback.	Encrypted loopback traffic.	Severe browser certificate warnings; modifies Windows Certificate Store.	REJECTED
Option C: Named Pipes Only	Eliminate HTTP port entirely; pass messages over Win32 Named Pipes.	Zero network attack surface.	Completely breaks OpenAI API client compatibility (/v1/chat/completions).	REJECTED

Decision 4: Air-Gapped Network & Offline UI Resilience

Approach	Technical Concept	Pros & Strengths	Cons & Trade-Offs	Verdict
Option A: Bundled Assets + GetAdapters	Vendor all JS/CSS/fonts locally; resolve IPs via iphlpapi.dll.	100% air-gapped functionality; zero CDN stalls; native adapter accuracy.	Adds 1.8 MB to static asset payload.	SELECTED
Option B: Dynamic CDN Fallback	Load from CDN; fallback to local files on script onerror.	Saves initial distribution bytes if online.	Causes 3-5 second DNS timeout stalls on air-gapped networks; layout shifts.	REJECTED
Option C: Dual Build Modes	Produce separate online and offline editions.	Tailored bundles.	Double maintenance overhead; high risk of feature divergence.	REJECTED

Decision 5: Self-Bootstrapping Engine Architecture

Approach	Technical Concept	Pros & Strengths	Cons & Trade-Offs	Verdict
Option A: Script Wrapper Runner	Bundle PowerShell scripts into a compiled executable runner.	Reuses existing script code.	Blocked by enterprise ExecutionPolicy; antivirus heuristics flag wrappers.	REJECTED
Option B: Native Rust Bootstrap	Perform DLL validation, PATH setup, secret creation, and Job init in Rust.	Instant sub-100ms launch; zero script dependencies; type-safe diagnostics.	Requires ~300 lines of robust systems bootstrap code.	SELECTED
Option C: Background Windows Service	Install background service as LocalSystem to manage prerequisites.	Can perform elevated actions silently.	Unjustified privilege elevation; corporate IT security blocker.	REJECTED

Decision 6: First-Run Onboarding Wizard & Model Management

Approach	Technical Concept	Pros & Strengths	Cons & Trade-Offs	Verdict
Option A: Terminal Prompt	Prompt user in console before launching window.	Simple text code.	Breaks desktop GUI immersion; confusing for non-technical users.	REJECTED
Option B: 3-Step Webview Wizard	In-app modal: 1. Environment, 2. Opt-in models with SSE progress, 3. Role.	Zero forced downloads; skip button for existing files; live download progress.	Requires authoring modal components and SSE download endpoint.	SELECTED
Option C: Settings Store Catalog	Drop user into blank chat; hide models in settings.	No modal overlay needed.	High cognitive friction; leaves first-time user stranded with non-working chat.	REJECTED

3. Engineering Decision Log (EDL-01 to EDL-11)

ID	Domain	Choice Made	Technical Justification
EDL-01	Lifecycle & Zombies	Option B: SafeProcessJob + Watchdog	Guarantees kernel-enforced process and VRAM release upon window close.
EDL-02	Windows Storage	Option B: %APPDATA% / %LOCALAPPDATA%	Eliminates ERROR_ACCESS_DENIED; enables unprivileged standard user operation.
EDL-03	Localhost Security	Option A: CSPRNG Token + Host Guard	Neutralizes drive-by browser exploits and DNS rebinding attacks on 127.0.0.1.
EDL-04	Air-Gapped Network	Option A: iphlpapi!GetAdaptersAddresses	Resolves local network interfaces accurately without outbound 8.8.8.8 UDP probes.
EDL-05	Offline UI Delivery	Option A: Bundled local vendor assets	Eliminates all third-party CDN latency and guaranteed rendering failures offline.
EDL-06	DiT Diffusion Stream	Option A: Real stdout parsing	Replaces synthetic 500ms sleep loops with truthful progress and instant OOM reporting.
EDL-07	High-DPI Display	Option A: Win32 PerMonitorV2 Manifest	Prevents DWM bitmap stretching and blur on multi-monitor mixed-DPI configurations.
EDL-08	Single-Instance Guard	Option A: Named Win32 Mutex	Prevents duplicate instance launches and avoids cascading port drift (7860 to 7861).
EDL-09	Diagnostic Logging	Option A: Rolling file + Minidump filter	Persists crash logs to disk; writes .dmp minidumps for post-mortem C++ analysis.
EDL-10	Self-Bootstrapping	Option B: Native Rust Runtime Bootstrap	Eliminates setup.ps1/start.ps1; provisions keys, verifies DLLs, and sets PATH autonomously.
EDL-11	Model Management	Option B: 3-Step Opt-In Onboarding Wizard	Strictly enforces zero forced downloads; provides lightweight starter models or instant skip.

4. Implementation Phasing & Task Dependency Structure

Work is organized into six strictly ordered tiers to prevent rework and guarantee continuous stability:

- **Phase 1: Foundation, Storage & Manifest Compliance:** AppData directory resolution, Win32 manifest, single-instance mutex, and rolling disk logging.
- **Phase 2: Security Hardening & Secret Cryptography:** CSPRNG token generation, localhost origin locking, and DNS rebinding host guard.
- **Phase 3: Kernel Job Trapping & Window Lifecycle:** SafeProcessJob integration in desktop binary and Edge window supervisor watchdog.
- **Phase 4: Air-Gapped Network & Systems Hardware:** Native Win32 adapter enumeration and typed ARP neighbor discovery.
- **Phase 5: Self-Bootstrapping Engine, Zero-Forced Downloads & Onboarding Wizard:** Autonomous dependency verifier, SSE downloader API, bundled offline UI assets, atomic chat persistence, and the 3-step Kimi dark matte wizard.
- **Phase 6: Distribution Packaging & Verification Gate:** Inno Setup installer script with clean uninstallation, full test suite validation, and release sign-off.

5. Detailed Step-by-Step Execution Plan (All 16 Tasks)

Task 1.1 [TSK-FND-01]: Unified Windows Storage Manager

CRITICAL • FOUNDATION

Technical Decision:	Implement dedicated path resolution using `dirs = "5.0"` to route settings to `%APPDATA%\AeroMESH` and runtime data to `%LOCALAPPDATA%\AeroMESH`.
Files Modified:	`crates/aeromesh-cli/Cargo.toml`, `crates/aeromesh-cli/src/bin/aeromesh_desktop.rs`, `crates/aeromesh-cli/src/main.rs`
Implementation:	1. Add `dirs = "5.0"` to Cargo dependencies. 2. Implement `get_config_dir()` (%APPDATA%), `get_data_dir()` (%LOCALAPPDATA%), and `get_logs_dir()`. 3. Refactor `ensure_env_secrets` to read/write `config.env` in AppData config dir. 4. Point Edge profile to `%LOCALAPPDATA%\AeroMESH\edge_profile` and images to `generated_images`. 5. Retain local repository fallback when running in developer mode.
Verification:	Launch binary from a read-only directory; verify zero `os error 5` permissions exceptions and confirm files populate AppData.

Task 1.2 [TSK-FND-02]: Win32 Application Manifest & Single-Instance Mutex

HIGH • FOUNDATION

Technical Decision:	Embed a Win32 application manifest declaring `PerMonitorV2` DPI awareness, Windows 10/11 GUIDs, and UTF-8; enforce single-instance via `CreateMutexW`.
Files Modified:	`crates/aeromesh-cli/build.rs`, `crates/aeromesh-cli/resources/aeromesh.manifest`, `crates/aeromesh-cli/src/bin/aeromesh_desktop.rs`
Implementation:	1. Create `aeromesh.manifest` with PerMonitorV2 DPI awareness and activeCodePage UTF-8. 2. Compile manifest into PE header via `winres` in build.rs. 3. At startup in main(), call `CreateMutexW(NULL, TRUE, L"Global\AeroMESH_Desktop_SingleInstance")`. 4. If ERROR_ALREADY_EXISTS, activate existing window and exit with code 0.
Verification:	Launch multiple instances concurrently; verify second instance detects primary and exits immediately without port drift.

Task 1.3 [TSK-FND-03]: Rolling File Logger & Crash Minidump Handler

MEDIUM • DIAGNOSTICS

Technical Decision:	Configure `tracing-appender` to write daily rolling logs to `%LOCALAPPDATA%\AeroMESH\logs\desktop.log` and install `SetUnhandledExceptionFilter` for minidump capture.
Files Modified:	`crates/aeromesh-cli/Cargo.toml`, `crates/aeromesh-cli/src/bin/aeromesh_desktop.rs`
Implementation:	1. Add `tracing-appender = "0.2"` to dependencies. 2. Configure tracing subscriber to write to both stdout and daily rolling file. 3. Register unhandled exception filter calling `MiniDumpWriteDump` from `dbghelp.dll` upon access violations.
Verification:	Trigger panic in debug build; confirm `desktop.log` contains full stack trace and a `.dmp` file is generated.

Task 2.1 [TSK-SEC-01]: CSPRNG Token Generation & Endpoint Authentication

CRITICAL • SECURITY

Technical Decision:	Generate 256-bit cryptographic session token via `BCryptGenRandom`; inject into HTML meta tag; enforce `Authorization: Bearer <token>` on all routes.
Files Modified:	`crates/aeromesh-cli/src/bin/aeromesh_desktop.rs`, `static/index.html`, `static/js/app.js`
Implementation:	1. Replace timestamp hash with BCryptGenRandom in ensure_env_secrets. 2. Inject token into served HTML via `<meta name="aeromesh-session-token" content="...">`. 3. Attach Bearer token header to all fetch requests in app.js. 4. Implement Axum middleware rejecting unauthenticated /api/* calls with 401 Unauthorized.
Verification:	Execute unauthenticated curl request to /api/node/start; verify 401 Unauthorized response.

Task 2.2 [TSK-SEC-02]: Localhost CORS Lockdown & Host Header Guard

CRITICAL • SECURITY

Technical Decision:	Restrict CORS allowed origins strictly to `http://127.0.0.1:<port>` and `http://localhost:<port>`; validate incoming Host headers to defeat DNS rebinding.
Files Modified:	`crates/aeromesh-cli/src/bin/aeromesh_desktop.rs`
Implementation:	1. Remove `allow_origin(Any)`; whitelist loopback origins only. 2. Implement middleware checking Host header against 127.0.0.1 and localhost, rejecting mismatches with 403 Forbidden.
Verification:	Simulate request from origin `https://attacker-domain.com`; verify browser preflight is blocked by CORS.

Task 3.1 [TSK-LFC-01]: Desktop SafeProcessJob Integration

CRITICAL • LIFECYCLE

Technical Decision:	Wire `SafeProcessJob` (`JOB_OBJECT_LIMIT_KILL_ON_JOB_CLOSE`) from aeromesh-engine into aeromesh_desktop.rs to trap all worker and diffusion processes.
Files Modified:	`crates/aeromesh-cli/src/bin/aeromesh_desktop.rs`
Implementation:	1. Instantiate `SafeProcessJob::new()?` in main() and store in DesktopState. 2. Assign child handles in `spawn_role_process` and `post_image_stream` immediately after `.spawn()`.
Verification:	Start Coordinator; kill AeroMesh-Desktop.exe via Task Manager. Verify aeromesh.exe terminates immediately and VRAM is freed.

Task 3.2 [TSK-LFC-02]: Native Window Lifecycle Watchdog & Coordinated Shutdown

CRITICAL • LIFECYCLE

Technical Decision:	Retain Edge child handle in `spawn_desktop_window`, monitor via `child.wait()` in dedicated thread, and signal graceful Axum server shutdown on window close.
Files Modified:	`crates/aeromesh-cli/src/bin/aeromesh_desktop.rs`
Implementation:	1. Return `Option<Child>` from spawn_desktop_window. 2. Spawn thread waiting on `child.wait()`, transmitting shutdown signal to tokio watch channel. 3. Pass shutdown signal to `axum::serve(listener, app).with_graceful_shutdown(...)`.
Verification:	Launch desktop app; click "X" on Edge window. Verify server logs clean exit and process terminates within 500 ms.

Task 4.1 [TSK-NET-01]: Native Win32 Adapter Enumeration (GetAdaptersAddresses)

HIGH • NETWORKING

Technical Decision:	Replace `8.8.8.8` UDP socket probe with direct Win32 `iphlpapi!GetAdaptersAddresses` API calls to discover active operational interfaces.
Files Modified:	`crates/aeromesh-cli/Cargo.toml`, `crates/aeromesh-cli/src/bin/aeromesh_desktop.rs`
Implementation:	1. Enable `Win32_NetworkManagement_IpHelper` in windows-sys. 2. Query adapters with GAA_FLAG_SKIP_DNS_SERVER, filtering for Ethernet and Wi-Fi adapters with OperStatusUp. 3. Extract primary unicast IPv4 address; fallback cleanly to 127.0.0.1 if disconnected.
Verification:	Disconnect WAN internet cable while keeping local Wi-Fi active; verify local LAN IP is accurately detected.

Task 4.2 [TSK-NET-02]: Win32 ARP Neighbor Enumeration (GetIpNetTable2)

MEDIUM • NETWORKING

Technical Decision:	Replace localized `arp -a` CLI parsing with typed Win32 `GetIpNetTable2` API calls.
Files Modified:	`crates/aeromesh-cli/src/bin/aeromesh_desktop.rs`
Implementation:	1. Call GetIpNetTable2 to retrieve typed MIB_IPNET_ROW2 entries. 2. Filter valid unicast IPv4 addresses and probe candidate worker nodes on port 50052.
Verification:	Run on Windows set to German or Japanese system display locale; confirm worker discovery succeeds.

Task 5.1 [TSK-BST-01]: Self-Bootstrapping Engine & Runtime Dependency Verifier

CRITICAL • BOOTSTRAPPING

Technical Decision:	Build autonomous bootstrap verifier inside `aeromesh_desktop.rs` that provisions keys, sets up PATH, audits CUDA DLLs, and checks VRAM without shell scripts.
Files Modified:	`crates/aeromesh-cli/src/bin/aeromesh_desktop.rs`
Implementation:	1. Implement `verify_and_bootstrap_environment()` checking bin/ and bin/diffusion/ DLLs. 2. Add bin directories to process PATH dynamically in-memory. 3. Provision config.env in %APPDATA%\AeroMESH. 4. Implement `GET /api/bootstrap/status` returning hardware and dependency readiness JSON. 5. Implement `POST /api/bootstrap/complete` writing `onboarding_complete.json`.
Verification:	Delete .env and launch AeroMesh-Desktop.exe directly (no setup scripts). Verify app initializes and serves UI cleanly.

Task 5.2 [TSK-BST-02]: Zero-Forced-Download Model Downloader API

CRITICAL • ZERO DOWNLOADS

Technical Decision:	Excise all automatic model curl scripts; implement opt-in SSE streaming download endpoint with byte progress, speed, ETA, and cancellation.
Files Modified:	`start.ps1`, `setup.ps1`, `app.py`, `crates/aeromesh-cli/src/bin/aeromesh_desktop.rs`
Implementation:	1. Remove automatic curl download of DreamShaper 8 from start.ps1 lines 396-401. 2. Define curated lightweight catalog in desktop binary (Qwen 2.5 1.5B ~0.98GB, Llama 3.2 3B ~2.02GB, DreamShaper 8 ~1.99GB). 3. Implement `POST /api/models/download` streaming SSE byte progress to client. 4. Implement `POST /api/models/download/abort` for clean cancellation.
Verification:	Launch clean workspace; verify zero network calls occur. Trigger download of Qwen 1.5B via API; verify SSE events report progress.

Task 5.3 [TSK-UI-01]: Offline Vendor Asset Packaging

HIGH • AIR-GAPPED UI

Technical Decision:	Bundle all vendor JavaScript libraries, stylesheets, and fonts locally in `static/vendor/` and update HTML references.
Files Modified:	`static/index.html`, `static/vendor/**`
Implementation:	1. Vendor pinned releases: Lucide (0.469.0), Marked (12.0.2), Marked-Highlight, DOMPurify (3.2.4), Highlight.js (11.9.0), KaTeX (0.16.9 + fonts). 2. Update index.html to reference local `/static/vendor/...` paths. 3. Ensure aeromesh_desktop.rs serves .woff2 font mime types correctly.
Verification:	Disconnect internet completely; confirm markdown, LaTeX math, icons, and code highlighting render without errors.

Task 5.4 [TSK-UI-02]: Atomic JSON Chat Storage in LocalAppData

HIGH • DATA PERSISTENCE

Technical Decision:	Provide `/api/conversations` endpoints writing atomically to `%LOCALAPPDATA%\AeroMESH\conversations.json`, with frontend write-through sync.
Files Modified:	`crates/aeromesh-cli/src/bin/aeromesh_desktop.rs`, `static/js/app.js`
Implementation:	1. Implement GET and POST /api/conversations in aeromesh_desktop.rs using atomic file renaming. 2. Update app.js to load conversations from backend and write-through on changes.
Verification:	Chat on port 7860; restart app on port 7861. Confirm all conversation history persists across port shifts.

Task 5.5 [TSK-UI-03]: Honest DiT Image Diffusion Progress & Stdout Telemetry

MEDIUM • TELEMETRY

Technical Decision:	Replace synthetic 500 ms sleep loop with real-time parsing of `sd-cli.exe` standard output.
Files Modified:	`crates/aeromesh-cli/src/bin/aeromesh_desktop.rs`, `static/js/app.js`
Implementation:	1. Pipe stdout from sd-cli.exe and stream genuine step progress via SSE. 2. Report true local metrics (0.0 MB Wire) for single-node execution. 3. Stream dit_error event immediately if sd-cli fails or OOMs.
Verification:	Trigger image generation without GPU model; verify immediate error event is rendered in UI without fake sleep ticks.

Task 5.6 [TSK-UI-04]: 3-Step First-Run Onboarding Wizard (Kimi Dark Matte + Lucide)

CRITICAL • USER EXPERIENCE

Technical Decision:	Implement 3-step modal wizard matching `liquid_glass.css` dark matte tokens, using Lucide SVG icons exclusively (zero emojis).
Files Modified:	`static/index.html`, `static/css/liquid_glass.css`, `static/js/app.js`
Implementation:	1. **Step 1 (Environment):** Auto-verify CUDA DLLs, GPU VRAM, credentials, and Tailscale/LAN IP with live checkmark cards. 2. **Step 2 (Opt-In Models):** Curated cards for Qwen 2.5 1.5B, Llama 3.2 3B, and DreamShaper 8 with live progress bar, plus prominent "Skip / I Have My Own Models" button. 3. **Step 3 (Role Launch):** Coordinator, Worker, or Local Mesh cards triggering launch and fading out wizard. 4. Enforce strict Kimi dark matte styling and zero emojis throughout.
Verification:	Launch clean app; verify wizard appears. Click "Skip" in Step 2; confirm zero downloads occur and Step 3 activates chosen role.

Task 6.1 [TSK-PKG-01]: Production Inno Setup Packaging Script

HIGH • DISTRIBUTION

Technical Decision:	Author Inno Setup script generating a signed single-file installer (AeroMESH-Setup.exe) with Start Menu shortcuts and clean uninstallation.
Files Modified:	`installer/aeromesh_setup.iss`
Implementation:	1. Bundle binaries, DLLs, and static assets targeting {autopf}\AeroMESH. 2. Register uninstaller with option to purge %LOCALAPPDATA%\AeroMESH. 3. Check for VC++ 2015-2022 redistributable and install silently if missing.
Verification:	Install on clean Windows VM; verify standard user launch and execute clean uninstaller.

6. Comprehensive Risk Register & Failure Mitigations

ID	Risk Description	Severity	Mitigation Strategy
RSK-01	Edge --user-data-dir blocked by Enterprise Group Policy	HIGH	Detect launch failure; fallback to system default browser via ShellExecuteW while notifying user.
RSK-02	Network disconnects during opt-in model download in Wizard	MEDIUM	Downloader writes to .tmp file and supports HTTP Range request resumption (-C - equivalent).
RSK-03	MoveFileExW atomic rename collision on Windows	MEDIUM	Implement retry loop with exponential backoff (up to 3 retries, 50ms) to accommodate file scanner locks.
RSK-04	Windows Defender SmartScreen blocks unsigned installer	HIGH	Document Authenticode signing workflow; submit installer to Microsoft Security Intelligence.
RSK-05	Missing VC++ Redistributable on bare Windows installation	HIGH	Configure Inno Setup installer to bundle and silently install vcredist_x64.exe if registry check fails.
RSK-06	User clicks "Skip" on Wizard with empty models folder	MEDIUM	Node Control Center clearly marks "No Local GGUF Found" and provides a folder picker / re-open wizard link.

7. Testing Plan & End-to-End Windows Validation Matrix

Test ID	Test Objective	Execution Method	Acceptance Criteria
TC-BST-01	Autonomous Bootstrap	Launch binary without running any .ps1 scripts	Binary provisions keys, verifies DLLs, and opens UI on port 7860.
TC-BST-02	Zero Forced Downloads	Launch app with empty models folder	Zero network calls to HuggingFace or curl; zero MB downloaded automatically.
TC-BST-03	Wizard Step 1	Observe Wizard Step 1 on first launch	CUDA DLLs, GPU VRAM, IPs, and credentials show verified badges with Lucide icons.
TC-BST-04	Wizard Step 2 (Download)	Select Qwen 1.5B and click Download in Step 2	Animated progress bar displays live MB, %, speed, and ETA; saves to models folder.
TC-BST-05	Wizard Step 2 (Skip)	Click "Skip / I Have My Own Models"	Immediately transitions to Step 3 without downloading any bytes.
TC-BST-06	Wizard Step 3 (Launch)	Select "Coordinator + UI" in Step 3	Backend starts coordinator, dismisses wizard, and enters Studio workspace.
TC-BST-07	Onboarding Persistence	Restart app after completing onboarding	App detects onboarding_complete.json and opens Node Control Center directly.
TC-WIN-01	Standard User Permissions	Run from C:\Program Files under standard account	App launches without os error 5; writes config to %APPDATA% and cache to %LOCALAPPDATA%.
TC-WIN-02	Zombie Elimination	Launch app, start worker, click "X" on Edge window	AeroMesh-Desktop.exe and child aeromesh.exe terminate within 500 ms; VRAM released.
TC-WIN-03	Single-Instance Guard	Double-click desktop shortcut twice	First window gains focus; second instance exits; no binding to port 7861.
TC-WIN-04	Cross-Origin Web Isolation	Execute fetch from external website console	Browser blocks request via CORS policy; server logs 403 Forbidden.
TC-WIN-05	Air-Gapped Startup	Disconnect network completely and launch app	App starts instantly; all icons, LaTeX math, and code highlighting render cleanly.

8. Rollback Protocol & Safeguards

To guarantee zero downtime and protect existing working code during remediation:

1. **Git Baseline:** Tag baseline as `CHECKPOINT_PRE_BOOTSTRAP`. Tag each completed task atomically (e.g. `CHECKPOINT_TASK_BST_01`).
2. **Onboarding Fallback:** Deleting `%LOCALAPPDATA%\AeroMESH\onboarding_complete.json` resets wizard state cleanly. Passing `?skip_wizard=1` in the URL bypasses the modal directly into the Node Control Center.
3. **Emergency Reversion Command:**

```
git reset --hard CHECKPOINT_PRE_BOOTSTRAP
cargo clean
cargo test --workspace
```

4. **Gate Condition:** All 92 automated tests (65 Rust + 27 Python) must pass without warnings at every phase boundary.

9. Final Ordered Task Execution Sequence

Dependency-Enforced Execution Path

1. [TSK-FND-01] Windows Storage Manager (%APPDATA% & %LOCALAPPDATA% Hierarchy)
2. [TSK-FND-02] Win32 Application Manifest & Single-Instance Mutex
3. [TSK-FND-03] Rolling File Logger & Unhandled Exception Minidump Filter
4. [TSK-SEC-01] CSPRNG Session Token Generation & Endpoint Authentication
5. [TSK-SEC-02] Localhost CORS Lockdown & Host Header Validation
6. [TSK-LFC-01] Desktop SafeProcessJob Integration (Win32 Job Object Limits)
7. [TSK-LFC-02] Native Window Lifecycle Watchdog & Coordinated Shutdown
8. [TSK-NET-01] Native Win32 Adapter Enumeration (iphlpapi!GetAdaptersAddresses)
9. [TSK-NET-02] Win32 ARP Neighbor Enumeration (GetIpNetTable2)
10. [TSK-BST-01] Self-Bootstrapping Engine (Autonomous Secrets, DLL Audit & Paths)
11. [TSK-BST-02] Zero-Forced-Download Model Downloader API (SSE Chunk Progress)
12. [TSK-UI-01] Offline Vendor Asset Packaging (Zero CDNs - Bundled JS/CSS/Fonts)
13. [TSK-UI-02] Atomic JSON Chat Storage in LocalAppData
14. [TSK-UI-03] Honest DiT Image Diffusion Progress & Stdout Telemetry
15. [TSK-UI-04] 3-Step First-Run Onboarding Wizard (Kimi Dark Matte + Lucide Only)
16. [TSK-PKG-01] Production Inno Setup Packaging Script (Signed Installer + Uninstaller)